

parallel way to find min

>

pagratios

Mar 29 '11

hello,

i have this code for finding the min value of an array (small sized).I run this in one block

```
nTotalThreads = blockDim.x;

while(nTotalThreads > 1)

{

    halfPoint = (nTotalThreads >> 1);

    // only the first half of the threa

    if (nTotalThreads%2 == 0 )

    {

        if (idx < halfPoint)

        {

            // Get the shared value sto

            temp = time_array[idx + hal

            if (temp < time_array[idx].t

            {

                time_array[idx].t = temp

                time_array[idx].pos = t

            }

        }

    }

    else

    {

        if (idx <= halfPoint)

        {

            // Get the shared value sto

            temp = time_array[idx + hal

            if (temp < time_array[idx].t

            {

                time_array[idx].t = temp

                time_array[idx].pos = t

            }

        }

    }

}

__syncthreads();

if (nTotalThreads%2 == 0 )
```

```
        nTotalThreads = (nTotalThreads
else
        nTotalThreads = (nTotalThreads

}
```

BUT when i run the code below (in serial program) is faster (i think we expect this because the size of the array is small)

```
for(i=0;i<N;i++)

{

    if(mint > akis[i])

        mint = akis[i];

}
```

What i have to do in order to accelerarate the parallel version???

avidday

Mar 29 '11

Use a parallel reduction. There is example code and a paper in the CUDA SDK. There are also convenient pre-written parallel reduction routines in the thrust template library.

pagratios

Mar 29 '11

I can't find it.
Also I want to find min between float values. Is the parallel algorith above a right one

YDD

Mar 29 '11

Reductions work on any associative operator. Of course, if the *in silico* operator isn't quite associative, results will vary slightly. However, that isn't the case for min.

avidday

Mar 30 '11

I can't find it.
It is called reduction and has been in every SDK release for roughly the last three years. There is also a version [here](#), and the thrust templates I mentioned are discussed [here](#).
Also I want to find min between float values.
A reduction is a general data parallel operation which can be applied using any associative operator.
Is the parallel algorith above a right one
I would not have thought so.

pagratios

Mar 30 '11

I use only one block to do it

avidday

Mar 30 '11

Even still, a parallel reduction will perform better than the bisection(?) approach you are using now. The key to performance is memory throughput, and something like this:

```
__shared__ volatile float buff[NTHREADS];

float minval = akis[threadIdx.x];

for(int i=blockDim.x + threadIdx.x; i<N; i+=blo

{

    minval = min(akis[i], minval);

}

buff[threadIdx.x] = minval;
```

```
__syncthreads();

// Do shared memory parallel reduction on buff
```

should perform much better than your current code or the naïve alternative.

pagratios Mar 30 '11

The problem is that I want not only the minimum value but the position in array.

Also I use only one block and array with values is small for example now I have an array with 19*sizeof(float).

LSChien Mar 30 '11

The problem is that I want not only the minimum value but the position in array.

The minimizer may not be unique, if there are two minimizers in position 2 and 100, which one do you want?

pagratios Mar 30 '11

The minimizer may not be unique, if there are two minimizers in position 2 and 100, which one do you want?

because my program is an algorithm for biology (Gillespie FRM method) I take the value in the time array from `time_array[idx] = (1/a)*logf(1/r)`, `r` is a random number from curand, it is difficult two position in array have the same value. But If have two same minimum value I don't care which one I'll take but I must know if it was in position 2 or 100.

Every thread is a reaction and I must know which reaction will run every time this is the reason why I have the `time_array` with structs type `data_t`.

```
typedef struct{

int pos;

float t;

}data_t
```

avidday Mar 30 '11

So the problem is really to find a blockwise minimum and record the position of the minimum of a data set where each data point is provided by a single thread? So the maximum number of points in the data set to search is the number of threads in a block?

pagratios Mar 30 '11

So the problem is really to find a blockwise minimum and record the position of the minimum of a data set where each data point is provided by a single thread? So the maximum number of points in the data set to search is the number of threads in a block?

I don't understand the "blockwise minimum".

The `time_array` (shared memory) has size of threads in block (reactions num).

avidday Mar 30 '11

Blockwise minimum just means the minimum of the value set over the block (as opposed to at some other scope like a global minimum).

If you already have the block data in a shared memory array, then the "tree" like shared memory reduction technique that has now been suggested three times is the most efficient way to reduce that to a blockwise minimum. Something like this:

```
__shared__ volatile data_t data[BLOCKSIZE];

// whatever your code does

__syncthreads();

// Here mindata(a,b) sets a to the minimum of a

if (threadIdx.x < 32) {

for(int i=32+threadIdx.x; i < BLOCKSIZE; i+= 32

mindata(data[threadIdx.x], data[i]);

}

if (threadIdx.x < 16) mindata(data[threadIdx.x]
```

```
    if (threadIdx.x < 8) mindata(data[threadIdx.x], data[threadIdx.x]);

    if (threadIdx.x < 4) mindata(data[threadIdx.x], data[threadIdx.x]);

    if (threadIdx.x < 2) mindata(data[threadIdx.x], data[threadIdx.x]);

    if (threadIdx.x == 0) mindata(data[0], data[0]);

}

// Blockwise minimum now held in data[0]
```

The volatile declaration is important if you are using a Fermi card, because there can be problems with compiler optimisation breaking the implicit warp level synchronization which the reduction relies on.

pagratios

Mar 30 '11

Blockwise minimum just means the minimum of the value set over the block (as opposed to at some other scope like a global minimum).

If you already have the block data in a shared memory array, then the “tree” like shared memory reduction technique that has now been suggested three times is the most efficient way to reduce that to a blockwise minimum. Something like this:

```
__shared__ volatile data_t data[BLOCKSIZE];

// whatever your code does

__syncthreads();

// Here mindata(a,b) sets a to the minimum of a

if (threadIdx.x < 32) {

    for(int i=32+threadIdx.x; i < BLOCKSIZE; i+= 32)

        mindata(data[threadIdx.x], data[i]);

}

if (threadIdx.x < 16) mindata(data[threadIdx.x], data[threadIdx.x]);

    if (threadIdx.x < 8) mindata(data[threadIdx.x], data[threadIdx.x]);

    if (threadIdx.x < 4) mindata(data[threadIdx.x], data[threadIdx.x]);

    if (threadIdx.x < 2) mindata(data[threadIdx.x], data[threadIdx.x]);

    if (threadIdx.x == 0) mindata(data[0], data[0]);

}

// Blockwise minimum now held in data[0]
```

The volatile declaration is important if you are using a Fermi card, because there can be problems with compiler optimisation breaking the implicit warp level synchronization which the reduction relies on.

My card isn't Fermi and is 1.1.

Also the mindata is command in cuda.h? In this code I find the minimum but what variable has the position of the original array? If block has more than 100 thead this code work?

avidday

Mar 30 '11

Also the mindata is command in cuda.h?

No. You write it for your data_t type.

In this code I find the minimum but what variable has the position of the original array?

The result is your own data_t. It contains the position and a value, doesn't it?

If block has more than 100 thead this code work?

Yes.

pagratios

Mar 30 '11

No. You write it for your data_t type.

Ok.

The result is your own data_t. It contains the position and a value, doesn't it?

Ok, because of mindata that will be.

Yes.

I'll try it and I hope to see few second less

pagratios

Apr 01 '11

No. You write it for your data_t type.

The result is your own data_t. It contains the position and a value, doesn't it?

Yes.

Previously I asked if it would work for 100 threads and answer yes but for 19 threads work?

avidday

Apr 01 '11

No. But why would you ever use less than 32 threads per block?

pagratios

Apr 01 '11

because every thread is a reaction for our algorithm and some biology models have less than 32 reactions

avidday

Apr 01 '11

17 threads per block is incredibly wasteful - 40% of every active warps CPU cycles aren't used and you will not be able to cover any of the fixed pipeline latency in the architecture. In real terms, you will be wasting about 90% of the GPU capacity.